

系统开发工具基础

第二次实验报告

命令行环境 开发环境与工具

丁翊君 学号：25020007016

2026 年 8 月 31 日

目录

1 实验目的

本次实验是《系统开发工具基础》课程的第二次实验，授课内容包括命令行环境、开发环境与工具、Debugging and Profiling 三部分。目前已完成前三题，主题如下：

1. **命令行环境：进程、信号与任务控制**：掌握 Shell 中后台任务的控制方法，理解进程信号与 trap 清理机制。
2. **开发环境与工具**：使用 IDE 语言服务器完成语义重构，借助 ruff 做静态检查，并用 pytest 验证代码正确性。
3. **Debugging**：使用调试器定位并修复归并排序中的索引缺陷。

2 实验环境

表 1: 实验环境配置

项目	配置
操作系统	Ubuntu 26.04.1 LTS (64 位)
Shell	GNU Bash 5.3.9
Python	3.14.4 (.venv 虚拟环境)
git	2.53.0
pytest	9.0.2
ruff	0.16.4
XeTeX	TeX Live 2025 / Debian
IDE	PyCharm / VS Code (启用 Python 语言服务器)

3 课上实验内容

3.1 第 5 题：控制一个可清理的后台任务

3.1.1 题目要求

将给定脚本保存为 q05/worker.sh，再完成任务控制。给定内容：

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
```

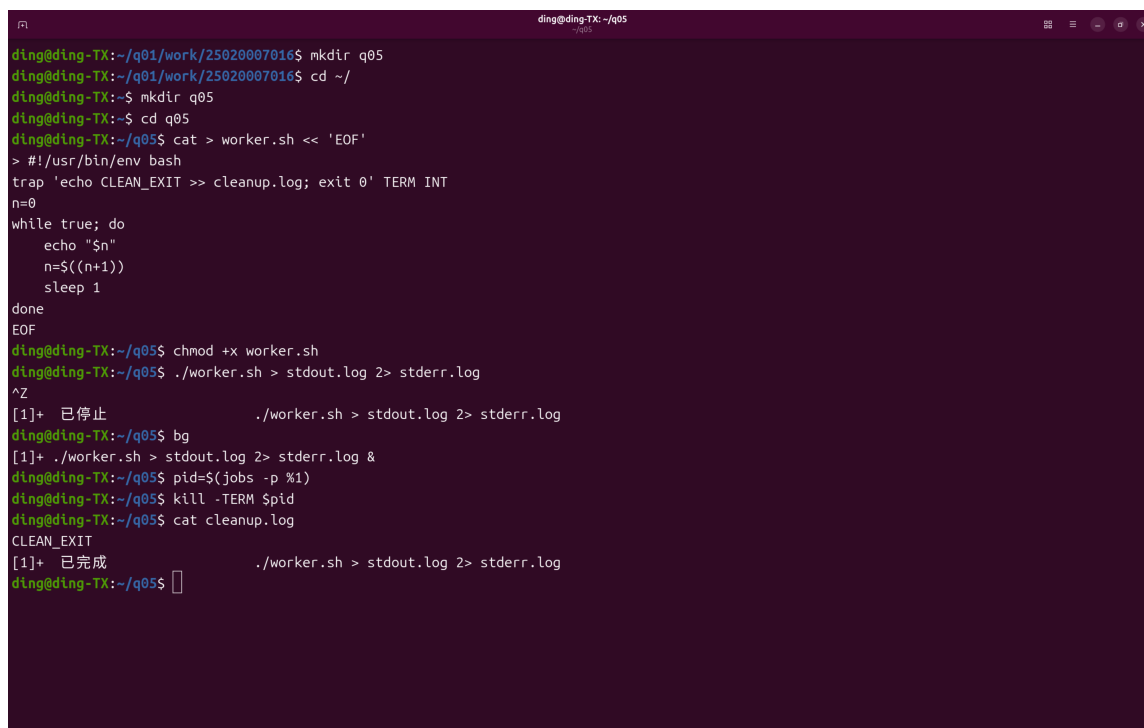
```
echo "$n"  
n=$((n+1))  
sleep 1  
done
```

题目要求：

- 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。
- 使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束。
- 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已经退出；禁止使用 kill -9。

3.1.2 执行过程与结果

1. 创建 q05 目录并编写 worker.sh，脚本通过 trap 捕获 TERM/INT 信号，收到信号时向 cleanup.log 写入 CLEAN_EXIT 后退出；chmod +x 赋予执行权限。
2. 前台启动脚本并把输出分离重定向：./worker.sh > stdout.log 2> stderr.log。
3. 按 Ctrl-Z 将任务挂起（输出 [1]+ 已停止），再用 bg 放入后台继续运行。
4. 用 pid=\$(jobs -p %1) 取得任务 PID（不手工抄写），执行 kill -TERM \$pid。
5. 查看 cleanup.log，出现 CLEAN_EXIT，任务状态变为已完成。



```
ding@ding-TX:~/q01/work/25020007016$ mkdir q05
ding@ding-TX:~/q01/work/25020007016$ cd ~/
ding@ding-TX:~$ mkdir q05
ding@ding-TX:~$ cd q05
ding@ding-TX:~/q05$ cat > worker.sh << 'EOF'
> #!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
ding@ding-TX:~/q05$ chmod +x worker.sh
ding@ding-TX:~/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log
ding@ding-TX:~/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
ding@ding-TX:~/q05$ pid=$(jobs -p %1)
ding@ding-TX:~/q05$ kill -TERM $pid
ding@ding-TX:~/q05$ cat cleanup.log
CLEAN_EXIT
[1]+  已完成                  ./worker.sh > stdout.log 2> stderr.log
ding@ding-TX:~/q05$
```

图 1: 第 5 题: 后台任务控制完整过程 (挂起、后台、取 PID、发送 SIGTERM)

3.1.3 关键技术点

1. **信号与 trap**: `trap '命令' TERM INT` 在收到 SIGTERM 或 SIGINT 时执行指定命令, 可以完成清理动作再退出; `kill -9 (SIGKILL)` 无法被捕获, 会直接杀死进程、跳过清理。
2. **任务控制**: `Ctrl-Z` 发送 SIGTSTP 挂起前台任务, `bg` 将其转为后台继续运行, `jobs` 查看任务列表与状态。
3. **获取 PID**: `jobs -p %1` 直接输出后台任务的 PID, 避免人工抄写出错。
4. **输出分离**: `> stdout.log 2> stderr.log` 分别重定向标准输出与标准错误, 便于排查。

3.2 第 6 题: 语义重构与本地开发反馈

3.2.1 题目要求

在 q06 中按照给定代码创建三个 Python 文件, 并使用编辑器语言服务器完成一次语义重构。给定内容:

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count
```

```
# app.py
from math_utils import total_price
print(total_price(12.5, 4))

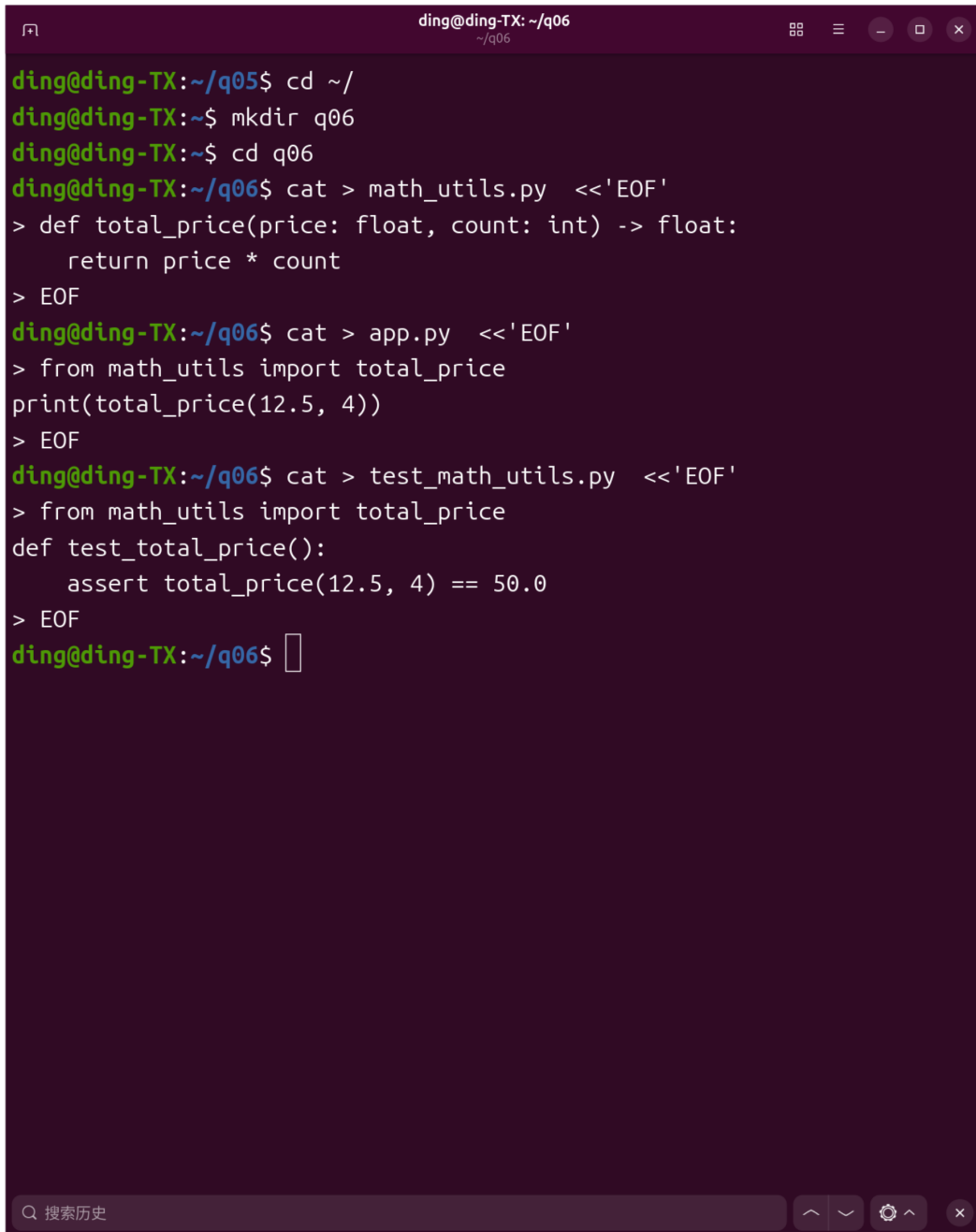
# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

题目要求：

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
- 分别演示跳转到定义和查找引用。
- 使用“重命名符号”把 `total_price` 改为 `calculate_total`，保证定义和所有代码引用同步改变。
- 在 `app.py` 中临时加入未使用的 `import os`，用 ruff 发现并删除；最后运行 ruff 和 pytest 并保证通过。

3.2.2 执行过程与结果

步骤 1: 创建三个 Python 文件。在 q06 下用 here-doc 写入 `math_utils.py`、`app.py`、`test_math_utils.py`。



```
ding@ding-TX: ~/q06
ding@ding-TX:~/q05$ cd ~/
ding@ding-TX:~$ mkdir q06
ding@ding-TX:~$ cd q06
ding@ding-TX:~/q06$ cat > math_utils.py <<'EOF'
> def total_price(price: float, count: int) -> float:
    return price * count
> EOF
ding@ding-TX:~/q06$ cat > app.py <<'EOF'
> from math_utils import total_price
print(total_price(12.5, 4))
> EOF
ding@ding-TX:~/q06$ cat > test_math_utils.py <<'EOF'
> from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
> EOF
ding@ding-TX:~/q06$
```

图 2: 第 6 题步骤 1: 创建三个 Python 文件

步骤 2: 语言服务器跳转与重命名。在 PyCharm 中启用 Python 语言服务器, 演示跳转到定义与查找引用; 随后使用“重命名符号”把模块 `math_utils` 重命名为 `calculate_total`, `app.py` 与 `test_math_utils.py` 中的 `import` 引用全部同步更新。

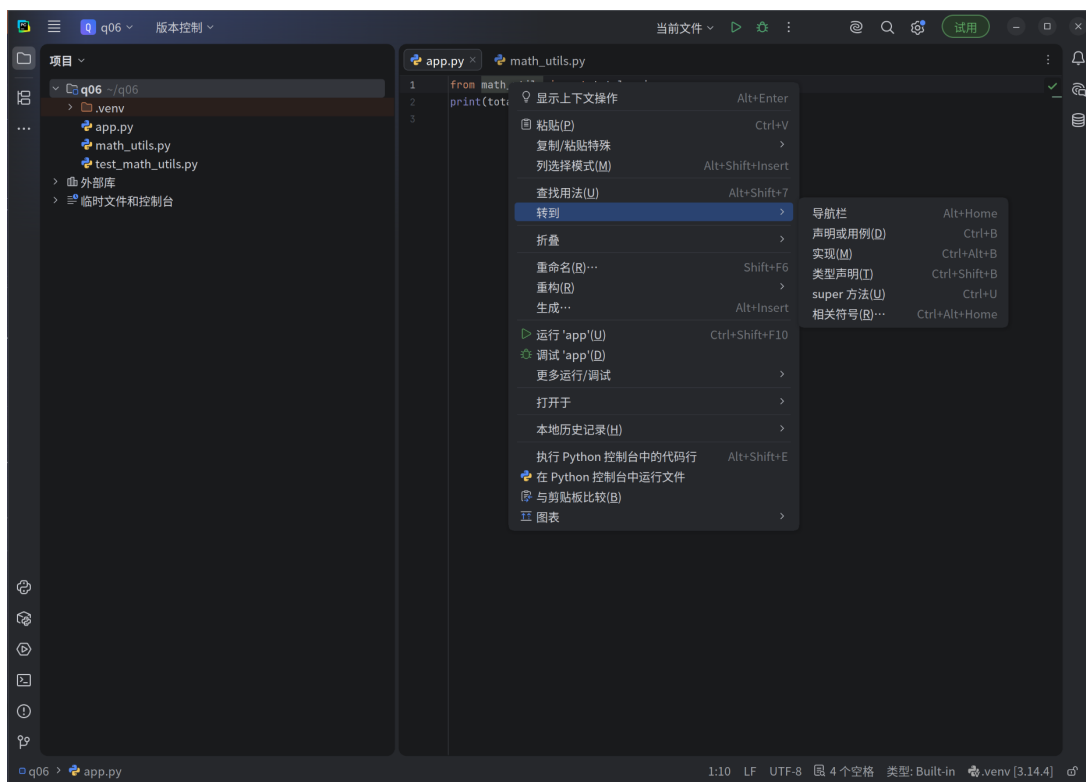


图 3: 第 6 题步骤 2: 语言服务器跳转到定义 / 查找引用

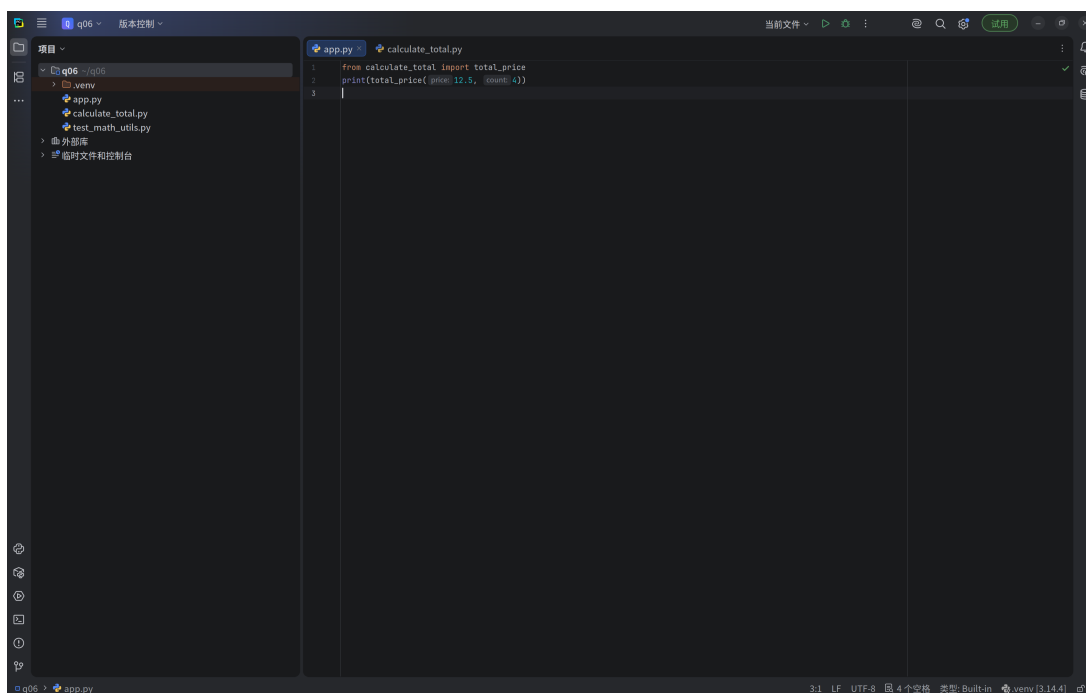


图 4: 第 6 题步骤 3: 重命名符号为 calculate_total (引用同步更新)

步骤 4: ruff 静态检查。在 `app.py` 临时加入未使用的 `import os`，安装 `ruff` 后执行 `ruff check`，报告 `I001`（导入块未排序/未格式化）与 `F401`（`os` 已导入但未使用），按提示删除该 `import`。

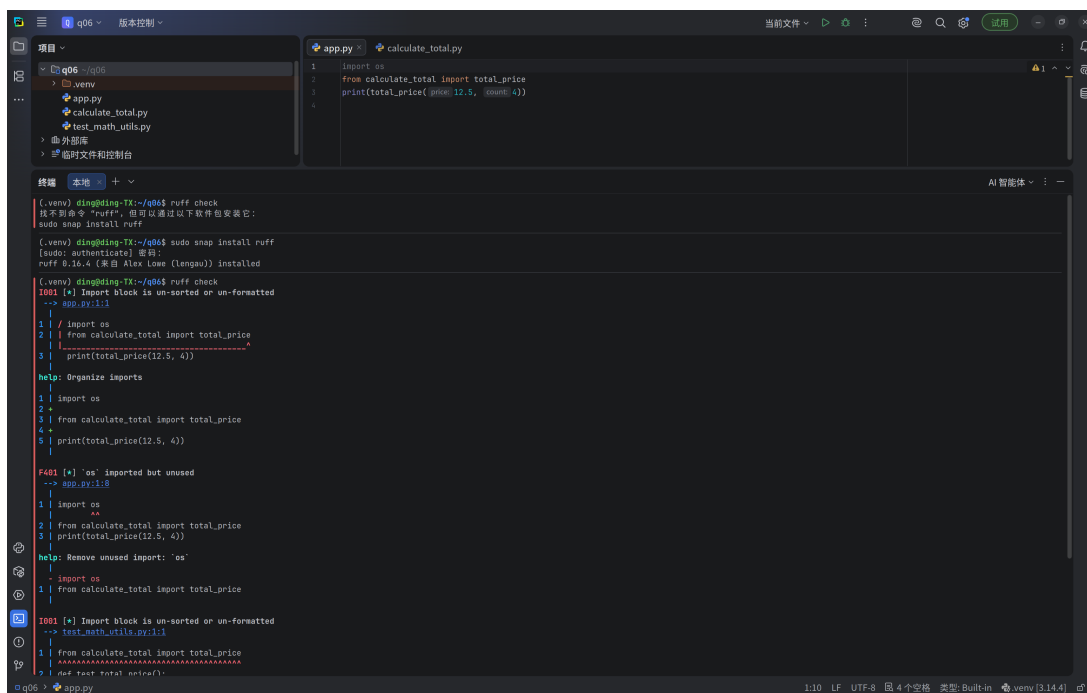


图 5: 第 6 题步骤 4: ruff 发现未使用 import os

步骤 5: 运行 pytest。安装 pytest 依赖后执行 pytest，收集到 1 个测试用例，test_math_utils.py 通过 (1 passed in 0.01s)。

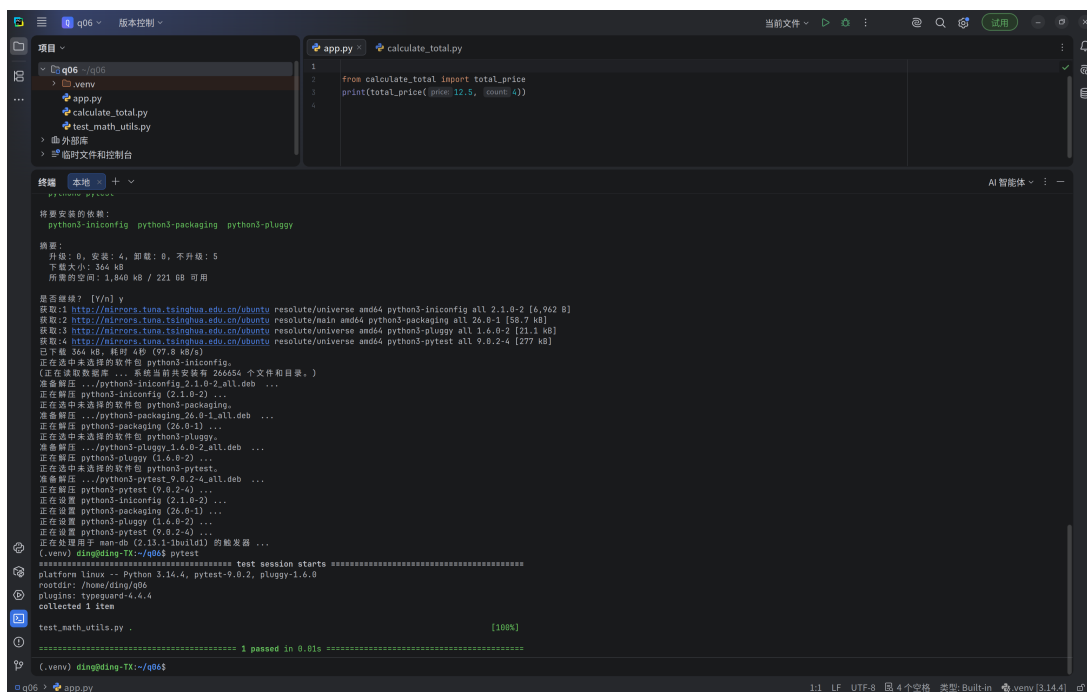


图 6: 第 6 题步骤 5: pytest 单元测试通过

3.2.3 关键技术点

1. **Python 语言服务器**：IDE 通过语言服务器获得跳转、引用、重命名等语义能力，而不是靠字符串匹配。
2. **重命名符号**：一次重命名同时更新定义与所有引用，避免手改遗漏造成不一致。
3. **ruff**：静态检查工具，F401 检查“导入但未使用”，I001 检查导入块排序与格式，可自动修复。
4. **pytest**：以 `test_` 前缀自动收集测试函数，断言不通过时给出失败信息，便于回归验证。

3.3 第 7 题：用调试器定位归并排序缺陷

3.3.1 题目要求

将下列存在缺陷的归并排序代码保存为 `q07/merge_sort.py`，再使用调试器定位并修复。给定内容：

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

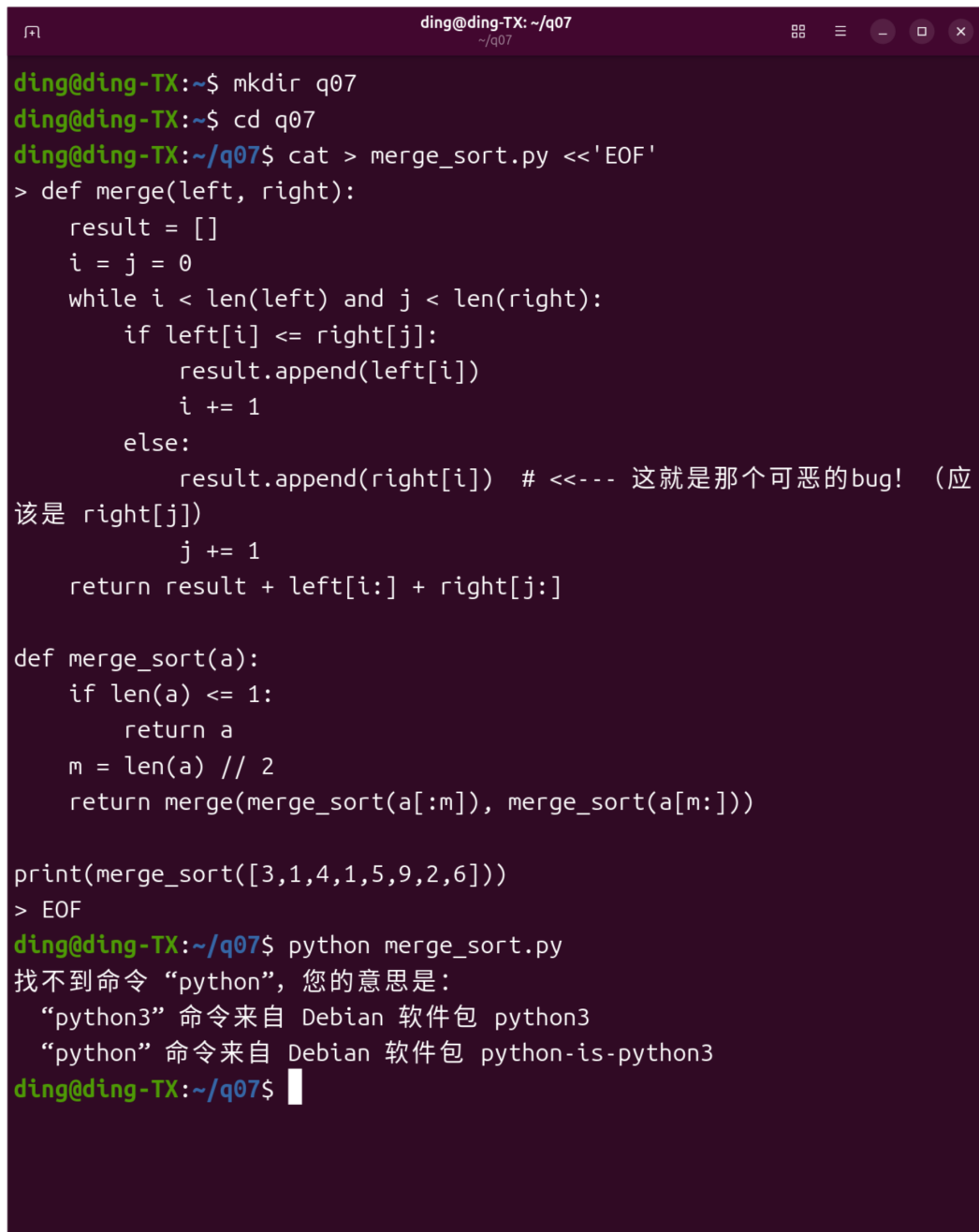
题目要求：

- 先运行给定输入，确认结果错误或出现异常。
- 使用 `pdb`、IDE debugger 或等价调试器，在 `merge` 中设置断点并观察 `i`、`j`、`left[i]`、`right[j]`。
- 完成最小修复，不得用 `sorted` 替换归并排序。

- 使用 pytest 增加两个测试：给定输入和含重复元素的列表；保证测试通过。

3.3.2 执行过程与结果

步骤 1：创建脚本并运行。在 q07 下创建 merge_sort.py（含缺陷代码），先运行确认结果异常。



```
ding@ding-TX: ~/q07
ding@ding-TX:~$ mkdir q07
ding@ding-TX:~$ cd q07
ding@ding-TX:~/q07$ cat > merge_sort.py <<'EOF'
> def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[i]) # <<--- 这就是那个可恶的bug! (应该是 right[j])
            j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3,1,4,1,5,9,2,6]))
> EOF
ding@ding-TX:~/q07$ python merge_sort.py
找不到命令“python”，您的意思是：
“python3”命令来自 Debian 软件包 python3
“python”命令来自 Debian 软件包 python-is-python3
ding@ding-TX:~/q07$
```

图 7：第 7 题步骤 1：创建含缺陷的 merge_sort.py

步骤 2：使用调试器定位缺陷。在 VS Code 中为 merge 设置断点，使用调试器 (debugpy) 单步执行，监视 i、j、left、right、result 的实时值。

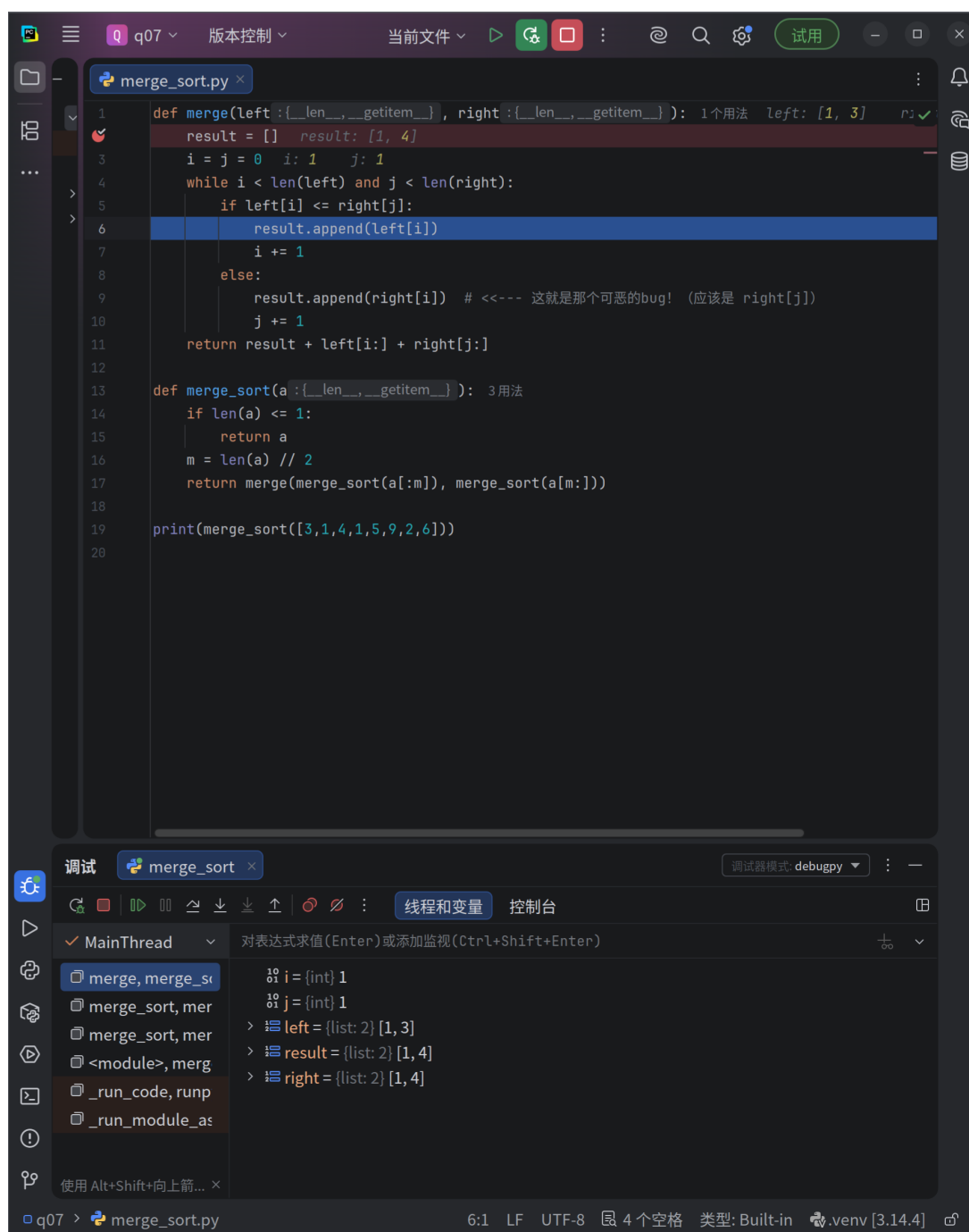
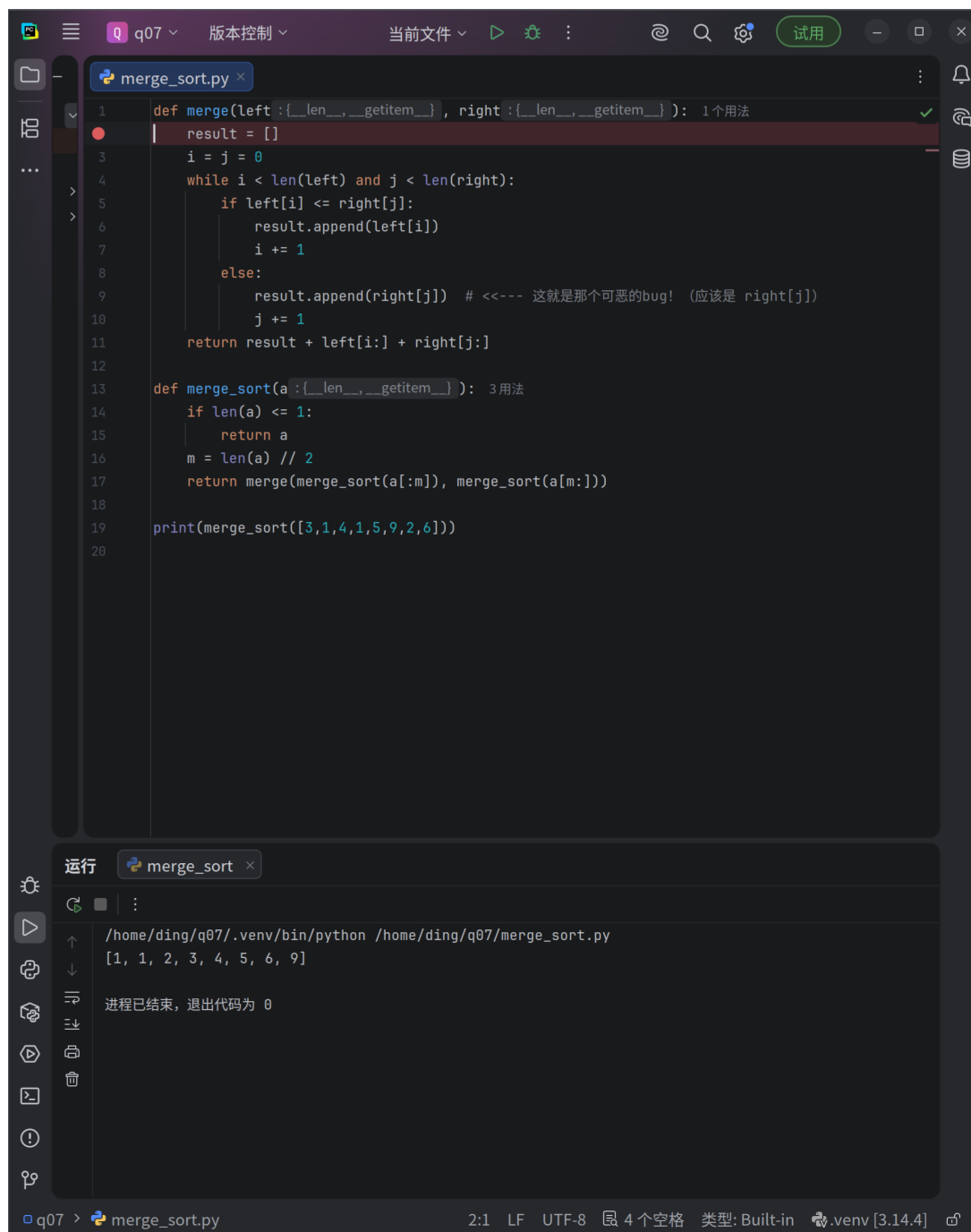


图 8: 第 7 题步骤 2: 断点调试观察变量

调试中发现: 当 `left=[1,3]`、`right=[1,4]`、`i=1`、`j=1` 时, `else` 分支执行 `result.append(right[i])` 用另一列表的下标 `i` 去索引 `right`, 定位到索引错误——本应取 `right[j]`。

步骤 3: 最小修复并验证。 将 `right[i]` 改为 `right[j]`, 重新运行, 输出正确结果 `[1, 1, 2, 3, 4, 5, 6, 9]`, 退出码 0。



The screenshot shows a code editor with a file named `merge_sort.py`. The code implements a merge sort algorithm. The `merge` function takes two lists, `left` and `right`, and merges them into a sorted `result` list. The `merge_sort` function recursively sorts a list `a`. The execution output at the bottom shows the sorted list `[1, 1, 2, 3, 4, 5, 6, 9]`.

```
def merge(left: {__len__, __getitem__}, right: {__len__, __getitem__}): 1个用法
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j]) # <<--- 这就是那个可恶的bug! (应该是 right[j])
            j += 1
    return result + left[i:] + right[j:]

def merge_sort(a: {__len__, __getitem__}): 3用法
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3,1,4,1,5,9,2,6]))
```

运行 `merge_sort`

```
/home/ding/q07/.venv/bin/python /home/ding/q07/merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
进程已结束, 退出代码为 0
```

图 9: 第 7 题步骤 3: 修复后运行输出正确排序结果

步骤 4: pytest 单元测试。新建 `test_merge_sort.py`, 按题目要求增加两个测试: 给定输入和含重复元素的列表。

```
from merge_sort import merge_sort

def test_merge_sort_given_input():
    # 给定输入
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5,
```

```
        6, 9]

def test_merge_sort_with_duplicates():
    # 含重复元素
    assert merge_sort([5, 2, 8, 2, 9, 1, 5, 5]) == [1, 2, 2, 5, 5, 5,
        8, 9]
```

执行 pytest，收集到 2 个测试用例，全部通过：

```
test_merge_sort.py::test_merge_sort_given_input PASSED    [ 50%]
test_merge_sort.py::test_merge_sort_with_duplicates PASSED [100%]
===== 2 passed in 0.22s =====
```

3.3.3 关键技术点

1. **断点调试**：在可疑分支设置断点、单步跟踪，比反复打印更能直观定位状态错误。
2. **变量监视**：调试面板实时显示 `i/j/left/right/result`，能立刻发现下标错位。
3. **最小修复**：只修改出错的一行，不重写算法，符合“最小改动”原则。
4. **pytest 回归**：以 `test_` 前缀自动收集测试函数，两个用例覆盖给定输入与含重复元素场景，防止修复引入回归。

3.4 第 8 题：先测量，再优化慢速词频程序

3.4.1 题目要求

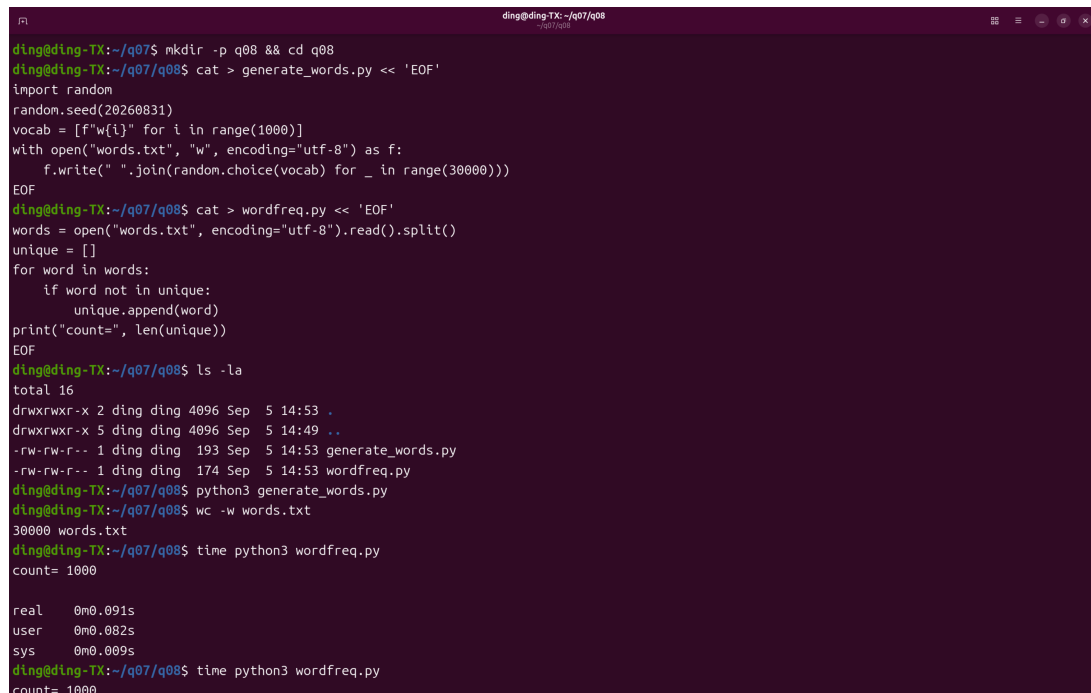
在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化：

1. 生成 `words.txt`，使用 `time` 运行原程序 2 次并记录耗时。
2. 使用 `cProfile -s cumulative` 运行一次，找出主要耗时位置。
3. 在不改变最终 `count` 的前提下优化程序；不得写死 1000。
4. 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

3.4.2 执行过程与结果

步骤 1：创建文件并生成数据。在 q08 下创建 `generate_words.py`（固定 seed 20260831，词表 1000，生成 30000 个词）和原始版 `wordfreq.py`（列表 + in 线性扫描

去重，整体 $O(n^2)$ 。运行 `generate_words.py` 生成 `words.txt`，再用 `time` 运行原始程序 2 次。



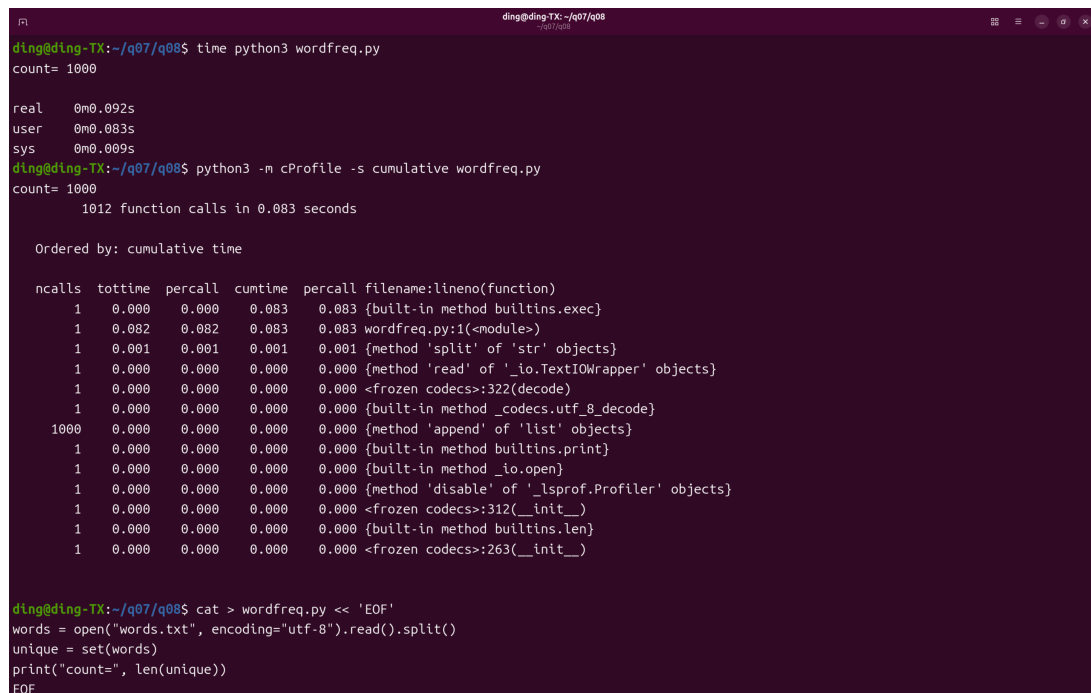
```
ding@ding-TX:~/q07/q08$ mkdir -p q08 && cd q08
ding@ding-TX:~/q07/q08$ cat > generate_words.py << 'EOF'
import random
random.seed(20260831)
vocab = [f'w{i}' for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
EOF
ding@ding-TX:~/q07/q08$ cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF
ding@ding-TX:~/q07/q08$ ls -la
total 16
drwxrwxr-x 2 ding ding 4096 Sep  5 14:53 .
drwxrwxr-x 5 ding ding 4096 Sep  5 14:49 ..
-rw-rw-r-- 1 ding ding 193 Sep  5 14:53 generate_words.py
-rw-rw-r-- 1 ding ding 174 Sep  5 14:53 wordfreq.py
ding@ding-TX:~/q07/q08$ python3 generate_words.py
ding@ding-TX:~/q07/q08$ wc -w words.txt
30000 words.txt
ding@ding-TX:~/q07/q08$ time python3 wordfreq.py
count= 1000

real    0m0.091s
user    0m0.082s
sys     0m0.009s
ding@ding-TX:~/q07/q08$ time python3 wordfreq.py
count= 1000
```

图 10: 第 8 题步骤 1: 创建文件、生成数据、优化前计时 (0.091s / 0.092s, count=1000)

原始程序输出 `count= 1000`，两次耗时分别为 0.091s 和 0.092s。

步骤 2:cProfile 定位瓶颈。运行 `python3 -m cProfile -s cumulative wordfreq.py`，按累计时间排序。输出显示主要耗时集中在模块级代码 (`wordfreq.py:1(<module>)`，`cumtime 0.082s`)，其中 `for` 循环内的 `if word not in unique` 列表线性扫描是瓶颈——30000 次迭代，每次平均扫描数百个已收集的唯一词。



```

ding@ding-TX:~/q07/q08$ time python3 wordfreq.py
count= 1000

real    0m0.092s
user    0m0.083s
sys      0m0.009s
ding@ding-TX:~/q07/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.083 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000   0.000   0.083    0.083 {built-in method builtins.exec}
1      0.082   0.082   0.083    0.083 wordfreq.py:1(<module>)
1      0.001   0.001   0.001    0.001 {method 'split' of 'str' objects}
1      0.000   0.000   0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000   0.000   0.000    0.000 <frozen codecs>:322(decode)
1      0.000   0.000   0.000    0.000 {built-in method _codecs.utf_8_decode}
1000    0.000   0.000   0.000    0.000 {method 'append' of 'list' objects}
1      0.000   0.000   0.000    0.000 {built-in method builtins.print}
1      0.000   0.000   0.000    0.000 {built-in method _io.open}
1      0.000   0.000   0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000   0.000   0.000    0.000 <frozen codecs>:312(__init__)
1      0.000   0.000   0.000    0.000 {built-in method builtins.len}
1      0.000   0.000   0.000    0.000 <frozen codecs>:263(__init__)

ding@ding-TX:~/q07/q08$ cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
EOF

```

图 11: 第 8 题步骤 2: cProfile 分析输出, 并重写为 set 优化版

步骤 3: 优化为 set 去重。将列表替换为 set, 哈希查找 $O(1)$, 整体复杂度降为 $O(n)$:

```

words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))

```

最终 count 由程序算出 (与优化前完全一致, 均为 1000), 未写死。

步骤 4: 优化后计时与加速比。使用相同输入再运行 2 次, 记录耗时并计算中位数和加速比。

```

ding@ding-TX:~/q07/q08$ cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
EOF
ding@ding-TX:~/q07/q08$ cat wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
ding@ding-TX:~/q07/q08$ time python3 wordfreq.py
count= 1000

real    0m0.022s
user    0m0.014s
sys     0m0.008s
ding@ding-TX:~/q07/q08$ time python3 wordfreq.py
count= 1000

real    0m0.016s
user    0m0.013s
sys     0m0.002s
ding@ding-TX:~/q07/q08$

```

图 12: 第 8 题步骤 4: 优化后计时 (0.022s / 0.016s), count=1000

性能对比:

表 2: 第 8 题优化前后性能对比

	优化前	优化后
第 1 次	0.091s	0.022s
第 2 次	0.092s	0.016s
中位数	0.0915s	0.0190s
count	1000	1000
加速比	—	4.82x

3.4.3 关键技术点

1. **先测量再优化**: 用 `time` 量化基线, 用 `cProfile -s cumulative` 按累计时间排序定位真正瓶颈, 不凭直觉改代码。
2. **数据结构选择**: 列表 `in` 是 $O(n)$ 线性扫描, `set` 是 $O(1)$ 哈希查找; 去重场景 `set` 是正确选择, 整体复杂度从 $O(n^2)$ 降为 $O(n)$ 。
3. **可重复实验**: 固定 `random.seed(20260831)`, 保证优化前后输入完全一致, 对比才有意义。
4. **正确性验证**: 优化前后 `count` 必须一致 (均为 1000), 加速比才有意义; 不得写死结果。